

Plan V2 : Système Solaire Temps Réel & Termineur Jour/Nuit

Status de développement

V2-7 (LUNES & ROVERS) —  **COMPLÉTÉ** (23 mars 2026)

-  **Lunes** : 20+ Lunes orbitant les planètes gazeuses + Terre
 - Composant `PlanetMoons.tsx` — orbites keplériennes avec inclinaison
 - Positions calculées keplerien ou via `astronomia` pour Lune/Io/Europa/etc
 - Textures WebP (`/textures/moons/`)
 - Tooltips au survol (nom, distance, période, description)
-  **Rovers** : Interface Mission Control pour les 5 rovers martiens
 - Composant `RoverOverlay.tsx` — overlay plein écran 3 colonnes (lazy-loaded)
 - Backend : endpoint `GET /api/rovers/positions` , table `rovers` , positions éditables en base
 - Composants : `RoverModel3D.tsx` (GLTF + Draco), `RoverPhotoGallery.tsx` (placeholder)
 - Intégration store Zustand : `roverPositions` , `fetchRoverPositions` , `selectedRoverId`
 - Marqueurs 3D sur Mars (composant `MarsRovers.tsx`) avec clic → overlay

État global : Mode système solaire fonctionnel avec planètes, lunes, rovers. Prêt pour V2-8+ (satellites, planètes en mode ciel).

Vision Générale

La V2 intègre une nouvelle fonctionnalité cohérente :

le **Système Solaire en temps réel**, visible depuis le globe et le ciel nocturne.

Le Soleil devenant un objet 3D positionné astronomiquement, le **termineur jour/nuit** en découle naturellement : la frontière jour/nuit sur le globe est tout simplement l'ombre portée par la direction réelle du Soleil — aucun calcul séparé requis. Toutes les positions se doivent d'être en temps réel.

Ambition V2 : Ce qu'on construit

```
MODE Système Solaire (vue de l'extérieur)
├─ Système Solaire en orbite autour du Soleil
│  ├─ Soleil (sphère lumineuse, PointLight)
│  ├─ Terre (notre globe actuel, en orbite)
│  ├─ 7 autres planètes (Mercury → Neptune)
│  └─ Lignes d'orbite elliptiques
│
├─ Termineur dynamique sur la Terre
│  ├─ Shader jour/nuit piloté par position 3D réelle du Soleil
│  └─ Zone de pénombre (crépuscule ~10°)
```

MODE GLOBE (vue de l'extérieur)

- └─ Clic sur une planète → Vue détaillée planète
 - └─ Zoom cinématique vers la planète
 - └─ Globe 3D texturé (style globe Terre actuel)
- └─ Retour au Système Solaire

MODE CIEL (vue depuis le sol)

- └─ Planètes visibles comme étoiles brillantes
 - └─ Taille proportionnelle à la magnitude apparente
 - └─ Couleur réaliste (Jupiter : beige, Mars : rouge...)
 - └─ Tooltip + panneau détails au clic
- └─ Soleil : visible à sa position exacte
 - └─ (avec avertissement si en dessous de l'horizon)

1. ÉVALUATION DES SOURCES DE DONNÉES

1.1 Calcul des positions planétaires

Trois approches possibles :

Approche	Précision	Dépendance externe	Complexité	Verdict
NASA JPL Horizons API	★★★★★ sub-arcseconde	API externe requise	Faible (fetch)	✗ Latence, rate limit
VSOP87 (série complète)	★★★★★ sub-arcseconde	Aucune	Haute (700 Ko tables)	⚠ Surdimensionné pour visuels
Meeus AA (algorithmie)	★★★★ ~1 arcmin	Aucune	Moyenne	✅ Retenu
Ephemeris simplifiée 3000BC-3000AD	★★★ quelques arcmin	Aucune	Faible	✅ Option B

Décision retenue : librairie npm `astronomia`

```
npm install astronomia
```

- Implémente les **Algorithmes de Jean Meeus** (Astronomical Algorithms, 2e éd.)
- Calcule RA/Dec + distance hélio/géocentrique pour les 8 planètes + Soleil + Lune
- 100% frontend TypeScript, zéro dépendance API
- Précision ~1 arcmin : **très largement suffisant pour la visualisation**
- Licence MIT

Alternative si `astronomia` insuffisant : `ephem.js` ou appel ponctuel à JPL Horizons lors d'une session (avec cache 1h).

1.2 Textures planétaires

Corps	Source	Résolution	Licence
-------	--------	------------	---------

Soleil	NASA Solar Dynamics Observatory	2048×1024	Domaine public
Mercure	NASA Messenger	2048×1024	Domaine public
Vénus (surface)	NASA Magellan	2048×1024	Domaine public
Mars	NASA Mars Reconnaissance Orbiter	4096×2048	Domaine public
Jupiter	NASA Cassini / JunoCam	4096×2048	Domaine public
Saturne + anneaux	NASA Cassini	2048×1024 + ring map	Domaine public
Uranus	NASA Voyager 2	1024×512	Domaine public
Neptune	NASA Voyager 2	1024×512	Domaine public

Toutes disponibles sur [Solar System Scope](https://www.solarsystemscope.com/textures/) (CC BY 4.0) ou NASA directement.

Source principale recommandée : <https://www.solarsystemscope.com/textures/>

- Toutes les planètes packagées, format JPG, qualité élevée, CC BY 4.0

2. STACK TECHNOLOGIQUE V2 (delta avec V1)

2.1 Nouvelles dépendances

```
# Calcul positions astronomiques (Meeus Algorithms)
npm install astronomia

# Optionnel : animations cinématiques de caméra
npm install @react-spring/three # si non déjà présent
```

2.2 Nouveaux composants frontend

```
frontend/src/
├── components/
│   ├── SolarSystem/
│   │   ├── SolarSystem.tsx      # [NEW] Conteneur principal du système solaire
│   │   ├── Planet.tsx          # [NEW] Planète générique (mesh + texture + orbite)
│   │   ├── Sun.tsx             # [NEW] Soleil (PointLight + surface animée)
│   │   ├── SaturnRings.tsx     # [NEW] Anneaux de Saturne
│   │   ├── OrbitLine.tsx       # [NEW] Ligne d'orbite elliptique
│   │   └── PlanetDetailView.tsx # [NEW] Vue globe planète au clic
│   ├── Globe.tsx               # [MODIFY] Shader terminateur jour/nuit
│   ├── NightSky.tsx            # [MODIFY] Planètes visibles comme étoiles
│   └── MilkyWay.tsx            # Inchangé
├── utils/
│   ├── skyCoords.ts            # Inchangé
│   ├── planetaryEphemeris.ts   # [NEW] Wrapper astronomia → positions 3D
│   └── dayNightShader.ts       # [NEW] GLSL shaders terminateur
```

```

|
├── stores/
|   └── useSolarSystemStore.ts      # [NEW] Zustand : état système solaire
|
└── types/
    └── planets.ts                  # [NEW] Types PlanetData, PlanetId, etc.

```

2.3 Modifications backend et Base de Données

Migration de SQLite vers PostgreSQL (Prod-Ready) : Pour une production robuste sur Railway, la base de données migrera de SQLite (actuel `nightsky.db` commité sur Git) vers une instance PostgreSQL gérée par Railway.

- Avantages : Plus de fichier binaire sur le dépôt Git, meilleures performances concurrentes, déploiements stateless.
- Implémentation : Script d'import initial sur la DB de prod au lieu de commiter le `.db`.

Pour le système solaire spécifiquement, **aucune modification métier backend requise** : les positions planétaires sont calculées entièrement côté frontend via `astronomia`. Le backend reste focalisé sur le catalogue d'étoiles (120k+) et les constellations.

3. ARCHITECTURE TECHNIQUE DÉTAILLÉE

3.1 Système de coordonnées

Le défi principal : **les échelles** du système solaire sont incompatibles avec l'affichage direct.

```

Réel :           Terre → Soleil : 149,6 millions km
                Terre → Neptune : 4,5 milliards km
Ratio Neptune/Terre : ~30

Affichage :      Globe Terre actuel : rayon = 1 unité THREE.js
                → Soleil serait à ~23 500 unités (impossible)

```

Solution : Deux modes d'affichage distincts

Mode "Vue Système Solaire" (nouvelle caméra grand angle)

- Échelle logarithmique ou compressée pour rendre toutes les planètes visibles
- Soleil au centre (0,0,0), Terre à ~5 unités, Neptune à ~50 unités
- Globe Terre agrandi visuellement pour rester lisible

Mode "Vue Planète" (actuel + nouvelles planètes)

- Zoom sur une planète individuelle → même expérience que le globe Terre actuel
- Chaque planète a sa propre scène Three.js ou un repositionnement de caméra

3.2 Pipeline de calcul des positions

```

// planetaryEphemeris.ts
import { solar, planetposition, base } from "astronomia";

interface PlanetPosition {
  ra: number; // Ascension droite (degrés) – pour mode ciel
  dec: number; // Déclinaison (degrés)

```

```

distance: number; // Distance géocentrique (UA)
helioLon: number; // Longitude héliocentrique (pour placement 3D)
helioLat: number; // Latitude héliocentrique
helioDist: number; // Distance au Soleil (UA)
// Coordonnées 3D Three.js (espace compressé)
position3D: THREE.Vector3;
}

function computePlanetPositions(
  timestamp: Date,
): Map<PlanetId, PlanetPosition> {
  // Calcul Julian Day Number
  const jde = dateToJDE(timestamp);
  // Calcul pour chaque planète via astronomia
  // Conversion coordonnées écliptiques → 3D Three.js (échelle compressée)
}

```

3.3 Terminateur jour/nuit — Shader GLSL

```

// dayNightShader.ts – Fragment shader pour Globe.tsx

uniform sampler2D dayTexture;    // Blue Marble (actuelle)
uniform sampler2D nightTexture; // Black Marble NASA
uniform vec3 sunDirection;      // Direction normalisée vers le Soleil (uniforme)
uniform float penumbraAngle;    // Largeur zone crépuscule (~10°)

varying vec3 vNormal;
varying vec2 vUv;

void main() {
  // Angle entre normale de surface et direction solaire
  float cosAngle = dot(normalize(vNormal), normalize(sunDirection));

  // Zone de pénombre : interpolation douce entre jour et nuit
  float blendFactor = smoothstep(-0.05, 0.15, cosAngle);

  vec4 dayColor = texture2D(dayTexture, vUv);
  vec4 nightColor = texture2D(nightTexture, vUv);

  // Mix : 0.0 = nuit totale, 1.0 = jour total
  gl_FragColor = mix(nightColor, dayColor, blendFactor);
}

```

`sunDirection` est mis à jour à chaque frame depuis la position 3D réelle du Soleil calculée par `astronomia` → le terminateur suit automatiquement la réalité.

3.4 Vue détaillée planète (clic → zoom)

Clic sur planète
→ Store : `selectedPlanet = "mars"`

→ Animation caméra (useFrame + lerp) : caméra → position face à la planète

→ PlanetDetailView :

- Même structure que Globe.tsx actuel
- Texture spécifique à la planète
- OrbitControls locaux
- Atmosphère si applicable (Mars rouge, Vénus dense, etc.)

→ Bouton "Retour" → animation inverse

4. SCÈNES ET MODES D'AFFICHAGE

4.1 Modes d'affichage

viewMode = "globe" (actuel)

→ [NEW] Sous-mode : "earth" (globe Terre seul, comportement V1)

→ [NEW] Sous-mode : "solar" (vue système solaire complet)

viewMode = "sky" (actuel, inchangé structurellement)

→ Planètes affichées comme points brillants à leur position réelle

4.2 UX de Rotation et Animation Temporelle

Pour équilibrer réalisme astronomique et expérience visuelle engageante, le comportement de rotation des corps dépend de la vitesse d'écoulement du temps :

- **Au repos (Temps Réel x1) : Réalisme contemplatif.** Les planètes tournent à leur vitesse réelle (elles semblent donc immobiles). Le Soleil pulse légèrement via un shader et la dérive lente des nuages (Terra) maintient la scène vivante.
- **Accélééré (Time Travel via le slider) : Animation spectaculaire.** En glissant le slider temporel, l'orbite des planètes et leur rotation propre s'accélèrent visuellement. Sur la Terre, le terminateur balaie la surface rapidement. Cette logique pilotée par `astronomia` ne nécessite aucun appel serveur, garantissant un 60 FPS parfait pendant l'accélération.

4.2 UI ajoutée

Bouton toggle : [🌍 Terre] ↔ [🌞 Système Solaire] (mode globe)

Panneau planète au clic :

- Nom + type (tellurique / géante gazeuse / géante de glace)
- Distance au Soleil (UA + km)
- Distance à la Terre actuellement (UA + km)
- Durée de révolution (jours/ans)
- Taille relative vs Terre
- Bouton "Explorer" → Vue détaillée globe

5. PERFORMANCES — ÉVALUATION & STRATÉGIE

5.1 Benchmarks attendus

Élément	Coût GPU (estimé)	Coût CPU
8 planètes (mesh sphère 64 segments)	~2 Mo VRAM	Négligeable

Textures planètes (8×2K WebP)	~30 Mo VRAM	Chargement initial
Shader terminateur Globe.tsx	+0.2ms/frame	Négligeable
Calcul astronomia positions	0 GPU	~2ms/calcul (1 fois/seconde)
Lignes d'orbite (LineLoop)	< 0.1 Mo	Négligeable
Anneaux Saturne (plane + texture)	~1 Mo	Négligeable

Objectif maintenu : 60 FPS constant en mode Vue Système Solaire.

5.2 Stratégies d'optimisation (Le défi des 30 Mo d'assets)

Le démarrage de l'application en mode "Système Solaire" requiert le chargement immédiat des textures planétaires (~30 Mo). Pour éviter un canvas noir, la stratégie suivante est adoptée :

- Loader 3D Global avec Suspense** : L'ensemble de la scène 3D initiale est enveloppé dans un `<Suspense>` avec un écran de chargement complet (barre de progression liée au statut de téléchargement des textures). L'univers ne s'affiche visuellement que lorsque tous les assets critiques de la vue de base sont prêts (0% de "pop-in" d'images blanches).
- Compression WebP Destructive qualitative (Lossy à 80%)** : Toutes les textures d'origine (JPG/PNG ultra-HD) doivent être converties en WebP avec perte (qualité `~80/100`). Les normales/specular maps pourront être divisées par deux en résolution (ex: 1024x512). Objectif : un poids total de ~10 Mo au démarrage.
- Preloading différencié** : Seules les *color/diffuse maps* base résolution sont bloquantes pour le loader initial. Les textures très haute définition utilisées uniquement lors du clic/zoom sur une planète individuelle (ex: nuages HD, normal map HD) seront préchargées en arrière-plan silencieusement (`useTexture.preload()`) *après* le démarrage de l'app.
- Throttle calcul positions** : `astronomia` interpole en temps réel lors du drag du slider. En x1, le calcul complet est throttlé (ex: 1/sec).
- LOD planètes lointaines** : Réduction drastique des segments de sphère (Uranus/Neptune → 16 segments).

5.3 Comparaison approches terminateur

Approche	Qualité	Perf	Complexité impl.
ShaderMaterial custom (retenu)	★★★★★	★★★★★	★★★
Mix deux Meshes superposées	★★★	★★★	★★
Texture générée canvas	★★★	★★	★★
API externe image	★★★★	★	★

6. PLAN DE RÉALISATION V2

SEMAINE V2-1 : FONDATIONS ASTRONOMIQUES

Objectif : Calcul des positions planétaires + tests de précision

Tâches :

- ☒ `npm install astronomia` + setup TypeScript types
- ☒ Créer `planetaryEphemeris.ts` : wrapper complet pour les 8 planètes + Soleil

- ☒ **Benchmark de précision** : vérifier positions vs données JPL Horizons pour 3 dates de référence
- ☒ Définir le système de coordonnées 3D compressé (mapping UA → unités Three.js)
- ☒ Créer `planets.ts` : types, constantes (rayons réels, couleurs, périodes)
- ☒ Créer `useSolarSystemStore.ts` : positions mises à jour 1 fois/seconde

Livable :

- Console log des 8 planètes + Soleil avec RA/Dec et position 3D toutes les secondes
- Précision vérifiée vs JPL Horizons (< 1° d'écart acceptable)

SEMAINE V2-2 : VUE SYSTÈME SOLAIRE — STRUCTURE 3D

Objectif : Afficher le système solaire navigable en mode globe

Tâches :

- ☒ Créer `Sun.tsx` (ou intégré) : sphère auto-lumineuse + `PointLight` + animation surface (bruit Perlin ou texture animée)
- ☒ Créer `OrbitLine.tsx` (ou intégré) : ellipse 3D (LineLoop) avec paramètres astrophysiques (demi-grand axe, excentricité)
- ☒ Créer `Planet.tsx` (ou intégré) : composant générique (mesh sphère, texture, rotation propre, raycasting clic)
- ☒ Créer `SaturnRings.tsx` (ou intégré) : plane circulaire avec texture d'anneaux + transparence
- ☒ Créer `SolarSystem.tsx` : assemblage + positionnement via store
- ☒ Toggle UI "Terre ↔ Système Solaire" dans `App.tsx` (via `SidePanel`)
- ☒ Adapter `OrbitControls` pour mode système solaire (limites de zoom différentes)

Livable :

- Vue système solaire navigable, planètes positionnées en temps réel, orbites visibles

SEMAINE V2-3 : TERMINATEUR JOUR/NUIT

Objectif : Shader terminateur sur le globe Terre piloté par position réelle du Soleil

Tâches :

1. Créer `dayNightShader.ts` : `vertexShader` + `fragmentShader` GLSL
2. Modifier `Globe.tsx` : remplacer `MeshStandardMaterial` par `ShaderMaterial`
 - Passer `dayTexture` (Blue Marble) + `nightTexture` (Black Marble) comme uniforms
 - Passer `sunDirection` depuis le store (mis à jour 1×/s)
 - Conserver : atmosphère shader existant (superposition additive)
3. ☒ Tester le terminateur à différentes dates/heures via le slider temporel
4. ☒ Ajuster la pénombre (zone crépuscule, largeur ~15°)
5. ☒ S'assurer que les normal maps (relief) fonctionnent toujours (à intégrer dans le ShaderMaterial)

Livable :

- Globe Terre avec frontière jour/nuit dynamique, lumières villes côté nuit, synchronisé avec le slider temporel

SEMAINE V2-4 : VUE DÉTAILLÉE PLANÈTE

Objectif : Clic sur une planète → exploration immersive

Tâches :

1. ☒ Créer la logique `selectedPlanet` et stocker ses propriétés physiques.
2. ☒ Gérer la transition caméra : animation spring depuis vue système solaire → face planète
3. ☒ Créer Panneau InfoCard : données planète (distance, masse, lune(s), température...)
4. ☒ Ajouter logique interactive (curseur pointeur/hover + onClick)
5. ☒ Ajouter Bouton "Retour au Système Solaire" dans la vue détaillée

Livable :

- Flow complet : Globe → Système Solaire → Clic planète → Vue globe planète → Retour
-

SEMAINE V2-5 : NOUVELLES FONCTIONNALITÉS (ROTATION, TERMINATEUR, ORBITES, ZOOM)

Objectif : Finalisation de la physique, des animations du système et de l'expérience visuelle.

Tâches :

- ☒ L'animation de la rotation du système (vitesses configurables, play/pause)
- ☒ Le réglage de bugs du terminateur
- ☒ L'ajustement des orbites (orbites au niveau du soleil)
- ☒ L'animation sur le zoom des planètes (hover scaling)
- ☒ Implémenter une représentation 3D de la ceinture d'astéroïdes réelle entre Mars et Jupiter (uniquement en mode système)
- ☒ Implémenter une représentation 3D de la ceinture de Kuiper (uniquement en mode système)

Livable :

- Une simulation du système solaire fluide, interactive, et performante.
-

SEMAINE V2-6 : AUDIT DE PERFORMANCES ET OPTIMISATIONS

Objectif : Fiabiliser et optimiser la version V2 pour éviter la surcharge processeur (CPU), réseau et vidéo (VRAM), garantissant une compatibilité avec les mobiles et PC modestes.

Tâches identifiées suite à l'audit du mode Globe Unifié :

- ☒ **1. [CPU] Optimisation de l'Éphéméride** :
 - *Statut* : **Résolu**. `computePlanetPositions` et `sunDirectionGlobal` sont désormais correctement mémorisés via `useMemo` dans `Globe.tsx` et ne se recalculent que lorsque le temps (`tsStr`) ou la planète sélectionnée change, évitant ainsi un appel à chaque frame.
- ☒ **2. [Réseau/Mémoire] Temporisation du prechargement dynamique** :
 - *Statut* : **Résolu**. `SolarSystem.tsx` utilise un ref `hoverTimeouts` avec un `setTimeout` de 350ms sur le `onPointerOver` , qui est annulé via `onPointerOut` . Cela évite les téléchargements inutiles lors d'un survol rapide.
- ☒ **3. [WebGL] Recompilation du Shader "DayNight"** :

- **Statut : Résolu.** Le code de `Globe.tsx` a été refactorisé pour utiliser `CustomShaderMaterial` au lieu de `onBeforeCompile`, et `dayNightShader.ts` exporte désormais des chunks GLSL séparés pour éviter les micro-saccades de compilation.

SEMAINE V2-7 : SATELLITES, ROVERS, COUVERTURE ORBITALE & ANNEAUX

Objectif : Enrichir les modes Globe et Système avec des objets orbitaux, des données d'exploration et de nouveaux corps planétaires.

Tâche 7.1 — Satellites naturels en mode Globe

Description : Afficher les principales lunes en orbite autour de leur planète lors du clic en mode Globe.

Lunes à implémenter :

- Terre : **Lune** (1 satellite)
- Jupiter : **Io, Europa, Ganymède, Callisto** (lunes galiléennes)
- Saturne : **Titan, Rhéa, Dioné, Téthys, Encelade** (5 principaux)
- Uranus : **Titania, Obéron, Umbriel, Ariel, Miranda** (5 principaux)
- Neptune : **Triton** (1 principal, orbite rétrograde)
- Mars : **Phobos, Deimos** (2 satellites)

Sources de données / calcul des positions :

- **Lune terrestre :** module `astronomia/moonposition` — calcul complet selon Meeus, précision ~1 arcmin, 100% frontend
- **Lunes galiléennes de Jupiter :** module `astronomia/jupitermoons` — positions des 4 lunes galiléennes selon Meeus Ch.44
- **Autres lunes :** pas de support natif dans `astronomia`. Deux options :
 - Option A (recommandée) : **positions statiques actualisées** depuis [JPL Horizons](https://ssd.jpl.nasa.gov/api/horizons.api) (`https://ssd.jpl.nasa.gov/api/horizons.api`) — appel API au montage du composant, cache 1h en mémoire
 - Option B : approximations Kepleriennes codées en dur (éléments orbitaux moyens depuis [NASA NSSDC Planetary Fact Sheets](https://nssdc.gsfc.nasa.gov/planetary/factsheets/))

Implémentation :

- Nouveau composant `PlanetMoons.tsx` monté uniquement si la planète sélectionnée a des satellites
- Chaque lune = petite sphère (radius ~0.05–0.2 selon taille réelle) texturée ou colorée
- Orbite visible (LineLoop en pointillés semi-transparents)
- Tooltip hover : nom + distance à la planète + période orbitale

Tâches :

- ☒ Implémenter `PlanetMoons.tsx` : rendu générique par planète — COMPLÉTÉ
 - ☒ Intégrer `astronomia/moonposition` pour la Lune terrestre — COMPLÉTÉ (éléments orbitaux statiques)
 - ☒ Intégrer `astronomia/jupitermoons` pour les 4 lunes galiléennes — COMPLÉTÉ (données Meeus statiques)
 - ☒ Appel JPL Horizons (ou éléments orbitaux statiques) pour les autres satellites — COMPLÉTÉ (statiques)
 - ☒ Textures lunes : [Solar System Scope](#) — COMPLÉTÉ (WebP dans `/textures/moons/`)
-

Tâche 7.2 — Rovers martiens : écran Mission Control plein écran

Description : Refactor complet de l'interaction rover. Au lieu d'un panneau latéral (`RoverInfoCard`), un overlay plein écran "Mission Control" descend du haut de l'écran avec animation slide-down. Layout 3 colonnes : modèle 3D rotatif | infos mission | galerie photos. Lazy-loaded pour la performance.

Note : L'API NASA Mars Rover Photos (`api.nasa.gov/mars-photos`) est **morte** (404 depuis fin 2025). Les photos seront fournies en statique ou via une future API alternative.

Rovers affichés :

Rover	Agence	Actif	Coordonnées approx.
Curiosity	NASA	✅ Oui (depuis 2012)	~137.4°E, 4.6°S (Gale Crater)
Perseverance	NASA	✅ Oui (depuis 2021)	~77.4°E, 18.4°N (Jezero Crater)
Opportunity	NASA	❌ 2004–2018	~354.5°E, 1.9°S (Endeavour Crater)
Spirit	NASA	❌ 2004–2010	~175.5°E, 14.6°S (Columbia Hills)
Zhurong	CNSA	❌ 2021–2022	~109.9°E, 25.1°N (Utopia Planitia)

Architecture :

1. Backend — Positions dynamiques :

- Endpoint `GET /api/rovers/positions` → retourne lat/lon de chaque rover
- Positions stockées en base (table `rovers`) ou fichier JSON, éditables sans redéploiement
- Le proxy photos NASA est **supprimé** (API morte)
- Architecture 4 couches : Model → Schema → Repository → Service → Router

2. Frontend — Overlay Mission Control :

- `RoverOverlay.tsx` (lazy-loaded via `React.lazy()` + `<Suspense>`) :
 - Overlay plein écran (`position: fixed` , `inset: 0` , `z-index: 1000`)
 - Animation slide-down CSS : `translateY(-100%)` → `translateY(0)` , `cubic-bezier(0.16, 1, 0.3, 1)` , 500ms
 - Bouton ✕ en haut à droite pour fermer (avec animation slide-up)
 - Layout 3 colonnes responsive :
 - **Gauche** : modèle 3D du rover (Canvas R3F isolé, `useGLTF` on-demand)
 - **Centre** : infos mission (nom, agence, dates, site d'atterrissage, description)
 - **Droite** : galerie photos (placeholder "Photos à venir" pour l'instant)
- `RoverModel3D.tsx` : placeholder (cube gris rotatif en attendant les GLTF)
- `RoverPhotoGallery.tsx` : grille vide avec message placeholder, prête pour `<img loading="lazy">`

3. Store Zustand :

- Ajout `roverOverlayClosing`: `boolean` (pilote l'animation de fermeture)
- Ajout `roverPositions`: `Record<string, {lat, lon}>` (cache positions backend)
- Ajout `fetchRoverPositions()` (fetch une fois, cache en store)

4. Types :

- `types/rovers.ts` : séparation métadonnées statiques (`ROVER_METADATA`) / positions dynamiques
- Ajout champs optionnels `modelPath?` , `photos?` pour enrichissement futur

5. Performance :

- `React.lazy()` : le bundle overlay n'est chargé qu'au premier clic rover
- Canvas R3F isolé dans l'overlay (pas de conflit avec le Canvas principal)
- `useGLTF` chargé on-demand (pas de preload de modèles 3D non visibles)
- Photos en `<img loading="lazy">` quand elles seront ajoutées
- Positions backend fetchées une seule fois et cachées dans le store

Tâches :

- ☒ Backend : modèle `Rover` + repository + service + endpoint `GET /api/rovers/positions` — COMPLÉTÉ
- ☒ Backend : supprimer le proxy photos NASA mort — COMPLÉTÉ (no photos endpoint)
- ☒ Store : ajouter `roverOverlayClosing` , `roverPositions` , `fetchRoverPositions` — COMPLÉTÉ
- ☒ `types/rovers.ts` : restructurer (séparer métadonnées statiques / positions dynamiques) — COMPLÉTÉ
- ☒ `RoverOverlay.tsx` : layout 3 colonnes + animation slide-down (lazy-loaded) — COMPLÉTÉ
- ☒ `RoverModel3D.tsx` : support GLTF + Draco decoder (cube placeholder fallback) — COMPLÉTÉ
- ☒ `RoverPhotoGallery.tsx` : placeholder grille vide "photos à venir" — COMPLÉTÉ
- ☒ `MarsRovers.tsx` : connecter aux positions dynamiques du store — COMPLÉTÉ
- ☒ `App.tsx` : intégrer `RoverOverlay` lazy-loaded — COMPLÉTÉ
- ☒ Cleanup : supprimer `RoverInfoCard.tsx` et le code backend NASA photos — COMPLÉTÉ

Tâche 7.3 — Couverture satellitaire en mode Globe (Terre uniquement)

Description : Visualisation des satellites en orbite autour de la Terre en temps réel. Composant activable/désactivable, disponible uniquement en mode Globe sur la Terre.

Données nécessaires :

1. TLE (Two-Line Elements) — format standard de description d'orbite satellite :

- Source : [Celestrak](https://celestrak.org) (gratuit, pas d'auth requise)
- Endpoints pertinents :

```
# Tous les satellites actifs (~6000 entrées)
https://celestrak.org/SOCRATES/query.php (complexe)

# Par catégorie (recommandé – moins de données) :
https://celestrak.org/SATCAT/tle.txt           # tous actifs
https://celestrak.org/supplemental/tle/starlink.txt # Starlink (~5000)
https://celestrak.org/TLE/table.php?GROUP=active&FORMAT=tle # actifs groupés

# Format JSON (plus pratique) :
https://celestrak.org/SATCAT/records.php?FORMAT=json
https://celestrak.org/TLE/format-new.php (GP format JSON)
```

- Les TLEs sont valides **~7–14 jours** → à re-fetcher régulièrement (cache backend recommandé)

- Recommandation : récupérer une **sélection thématique** plutôt que tout (ex: Starlink, GPS, météo, ISS+environnement proche)

2. Calcul des positions :

- `satellite.js` (SGP4 propagator) — **déjà dans le stack** (utilisé pour l'ISS)
- Calcul 100% frontend, aucun appel API en temps réel
- `satellite.propagate(satrec, date)` → position ECI → convertir en lat/lon/alt → coordonnées 3D sphériques
- Performance : SGP4 pour 500 satellites = ~2ms/frame → **throttler à 1 calcul/seconde** (positions ne changent pas visiblement plus vite)

Architecture recommandée :

- Nouveau composant `SatelliteCoverage.tsx`, monté si `selectedPlanet === "earth"` et toggle activé
- Bouton toggle dans le HUD Globe : `[☐ SATELLITES]`
- Au montage : fetch TLEs depuis CelesTrak (via backend pour CORS), parse avec `satellite.js`, `satrec` stocké en mémoire
- `useFrame` throttlé (1×/s) : propagation SGP4 → mise à jour `instancedMesh` (points/sphères minuscules)
- `instancedMesh` pour la performance (1000–5000 satellites simultanés)
- Code couleur par catégorie : Starlink (bleu), GPS (vert), météo (jaune), ISS+zone (cyan), débris (rouge)
- Tooltip hover : nom du satellite, altitude, vitesse, inclinaison

Tâches :

- ☒ Endpoint backend `GET /api/satellites/tle?group={group}` — proxy CelesTrak + cache TTL 12h — COMPLÉTÉ
- ☒ Créer `SatelliteCoverage.tsx` avec `instancedMesh` pour perf — COMPLÉTÉ
- ☒ Intégrer propagation SGP4 via `satellite.js` (déjà installé) — COMPLÉTÉ
- ☒ Toggle UI dans le HUD Globe (bouton activable/désactivable) — COMPLÉTÉ
- ☒ Throttle calcul positions à 1×/s avec scratch variables (règles R3F performance) — COMPLÉTÉ
- ☒ Sélection de groupes affichables (Starlink, GPS, ISS...) — COMPLÉTÉ
- ☒ Panneau infos satellite au clic — COMPLÉTÉ

Note : Les tâches 7.4 (anneaux Jupiter/Uranus/Neptune), V2-8 (planètes mode ciel) et V2-9 (polish & tests) ont été reportées en **V3**. Voir `docs/PLan-V3.md`.

8. VALIDATION & DÉPLOIEMENT V2

8.1 Checklist de validation avant merge sur `main`

Tests backend

```
cd backend
python -m pytest tests/ -v # tous les tests unitaires
python -m pytest tests/ --cov=app --cov-report=term-missing # couverture
```

- ☐ Tous les tests passent (0 failure, 0 error)
- ☐ Couverture > 80% sur `app/services/` et `app/repositories/`

- ☐ Endpoint `GET /api/rovers/positions` répond 200
- ☐ Endpoint `GET /api/satellites/tle?group=stations` répond 200
- ☐ Endpoint `GET /api/stars/visible` répond 200 avec cache TTL actif

Tests frontend

```
cd frontend
npx vitest run
```

- ☐ Tous les tests Vitest passent
- ☐ Pas d'erreur TypeScript : `npx tsc --noEmit`
- ☐ Pas d'erreur ESLint : `npx eslint src/`

Vérifications fonctionnelles manuelles

- ☐ Mode Globe — terminateur jour/nuit synchronisé avec le slider temporel
- ☐ Mode Globe — ISS visible et animée en temps réel
- ☐ Mode Globe — toggle Satellites actif, `instancedMesh` sans drop de FPS
- ☐ Mode Globe — clic Mars → overlay Rovers s'ouvre (slide-down), fermeture propre
- ☐ Mode Globe — lunes visibles au clic sur une planète (Jupiter, Saturne, Terre...)
- ☐ Mode Système Solaire — 8 planètes + Soleil positionnés, orbites visibles
- ☐ Mode Système Solaire — clic planète → panneau InfoCard + zoom caméra
- ☐ Mode Système Solaire — slider temporel accélère les orbites visuellement
- ☐ Mode Ciel — 5 000+ étoiles affichées, constellations tracées, ISS visible

Performances

- ☐ 60 FPS stable en mode Système Solaire (vérifier avec Chrome DevTools > Performance)
- ☐ Aucune fuite mémoire après 5 minutes en mode Système (heap stable)
- ☐ Taille bundle frontend < 2 Mo gzippé : `npm run build && npx vite-bundle-visualizer`

8.2 Préparation du merge

```
# Depuis la branche v2 - s'assurer que tout est committé
git status
git add <fichiers modifiés>
git commit -m "feat: finalisation V2 - lunes, rovers, satellites, terminateur"

# Récupérer les derniers changements de main
git fetch origin main
git rebase origin/main # ou merge selon la préférence

# Résoudre les conflits éventuels, puis :
git push origin v2
```

Ouvrir une Pull Request `v2 → main` sur GitHub :

- Titre : `feat: V2 - Système Solaire, terminateur, lunes, rovers, satellites`
- Description : lister les fonctionnalités livrées (V2-1 à V2-7.3)

- Reviewer : soi-même ou un pair

8.3 Déploiement

Le déploiement est **automatique** via les webhooks Railway (backend) et Vercel (frontend) déclenchés par le push sur `main`.

Étapes

1. **Merger la PR** `v2` → `main` sur GitHub (ou merge direct si solo)

```
git checkout main
git merge v2 --no-ff -m "Merge V2 : système solaire, terminateur, lunes, rovers, satellites"
git push origin main
```

2. **Vérifier le déploiement Railway (backend)**

- Dashboard : `railway.app` → projet → service backend
- Logs de build : `pip install -r requirements.txt` doit passer sans erreur
- Vérifier que la migration SQLite (ou seed PostgreSQL si applicable) s'est exécutée
- Health check : `curl https://<railway-url>/health` → `{"status": "ok"}`

3. **Vérifier le déploiement Vercel (frontend)**

- Dashboard : `vercel.com` → projet → dernier déploiement
- Build log : `npm run build` sans erreur TypeScript
- URL de preview Vercel disponible avant promotion en production
- Vérifier `VITE_API_URL` pointe bien vers l'URL Railway de production

4. **Smoke test en production**

- ☐ Page se charge sans erreur console (F12)
- ☐ Mode Globe opérationnel (terminateur visible)
- ☐ Mode Système Solaire opérationnel (planètes positionnées)
- ☐ Overlay Rovers accessible depuis Mars
- ☐ Endpoint satellites ne génère pas d'erreur CORS

Variables d'environnement à vérifier

Service	Variable	Valeur attendue
Vercel	<code>VITE_API_URL</code>	URL Railway prod (ex: <code>https://nightsky-api.up.railway.app</code>)
Railway	<code>DATABASE_URL</code>	URL PostgreSQL Railway (ou chemin SQLite si non migré)
Railway	<code>ALLOWED_ORIGINS</code>	URL Vercel prod (ex: <code>https://nightsky.vercel.app</code>)

8.4 Après déploiement

- ☐ Créer un tag Git de release : `git tag v2.0.0 && git push origin v2.0.0`
- ☐ Mettre à jour le `README.md` avec les nouvelles fonctionnalités V2
- ☐ Archiver les notes de `Plan-V2.md` (ce fichier reste en référence)

- ☐ Ouvrir la branche `v3` et commencer par V3-1 (Artémis 2) selon `Plan-V3.md`
-